

All You (Probably Not, But Maybe) Need is Logic (and DDTs) - Final Report

ZACK FREEMAN U1190843 & THOMAS KAUFFMAN U1374928

Reward learning has leveraged the advancements made in deep learning over the past decades, but with that comes decreased interpretability. We investigate how we can use Differentiable Decision Trees (DDTs) paired with Fuzzy Logic to create a novel training paradigm that leverages the logical relationship of pairwise preferences with the goal of ultimately enhancing the interpretability of a learned reward function.

1 Training the Differentiable Decision Tree as A Neuro-Symbolic Reward Model

1.1 Differentiable Decision Tree Formulation

We define a DDT as a binary tree consisting of internal nodes and leaf nodes, which aligns with the approach formalized in Kalra et al.'s paper [?] and originally shown in Frosst et al. [?]. While Kalra et al. propose multiple versions of these internal nodes, we will focus on the "simple" formulation of an internal node, in which each internal node consists of a linear neural layer, parametrized by w and b , where β is the inverse temperature parameter [?]:

$$\begin{aligned} P(\text{left}) &= \sigma(\beta(wx^T + b)) \\ P(\text{right}) &= 1 - \sigma(\beta(wx^T + b)) \end{aligned} \tag{1}$$

We parameterize each leaf with ϕ_i^l and define a probability distribution Q^l as the softmax over ϕ^l , i.e., $Q^l = \text{softmax}(\phi^l)$. The reward is then a min-max interpolation over the probability distribution, where the min and max reward are provided from a user and can be defined as $R = (R_{\min}, R_{\max})$. The interpolation is then a dot product: $R^l = Q^l \cdot R$.

1.2 Hard and Soft Routing

Finally, since this reward depends on all path probabilities through the tree, and therefore all parameters of internal nodes above the current leaf node, we have to create a reward signal that will meaningfully backpropagate through to each of these learnable parameters. With a DDT, there are two approaches: 1) a soft routing approach, where an expectation over rewards at the leaves is calculated, and 2) a hard routing approach, where the reward at the leaf with the maximum path probability is returned.

The soft routing approach is calculated as follows:

$$r_\theta(x) = \sum_l P^l(x)(Q^l R^l) \tag{2}$$

Where P^l is $P(l|I_n, \dots, I_1)$ and x is the current state of the episode in the environment.

For the hard routing approach, we can perform an argmax over the path probabilities to the leaf nodes and drop the summation:

$$r_\theta(x) = \text{argmax}_{pl(x)}(R^l) \tag{3}$$

2 Logic as Loss - Using T-Norms To Relax Logical Expressions

2.1 Bradley-Terry Annotation Constraint

For a pair of trajectories $\tau_1, \tau_2 \in T$, and a preference annotation from a user, we want the trajectory with the higher cumulative reward from each state action pair to correspond to the preference over the trajectories. Li et al. show that we can write "if an example is annotated with label Y^* (which is trivially true given our dataset), then the model should predict so" as $\top \rightarrow Y^*(\tau)$ [?]. Extending this idea to direct preference alignment, we propose the following formulation: *if a trajectory τ_+ is preferred over another trajectory τ_- , then the model should predict a higher reward for τ_+ than τ_-* . It follows that since the antecedent of this implication is trivially true, we can write the following:

$$\top \rightarrow (r_\theta(\tau_+) > (r_\theta(\tau_-))) \quad (4)$$

Furthermore, given each pair of trajectories, a preference for each pair, and a model output r_θ as defined in Equation 2—the cumulative reward for the episode,

$$\bigwedge_{\tau_+, \tau_- \in T} \top \rightarrow (r_\theta(\tau_+) > (r_\theta(\tau_-))) \quad (5)$$

2.1.1 Claim: *by relaxing the logical formula with the product t-norm [?], this implication recovers the cross-entropy loss for RLHF under the Bradley-Terry model.* We can now substitute our product T-norm given the tautology of the antecedent and relax the above expression to be simply the product of the probabilities of the preferred trajectory's reward being larger than the unpreferred:

$$\prod_{\tau_1, \tau_2, Y^* \in T} P(r_\theta(\tau_+) > (r_\theta(\tau_-))) \quad (6)$$

Under the Bradley-Terry model, this probability is:

$$P(R_\theta(\tau_+) > (R_\theta(\tau_-))) = \frac{\exp(\sum_{x \in \tau_+} r_\theta(x))}{\exp(\sum_{x \in \tau_+} r_\theta(x)) + \exp(\sum_{x \in \tau_-} r_\theta(x))} \quad (7)$$

And substituting the probabilities back into the product, and transferring to the negative-log space:

$$\begin{aligned} L_{BT} &= -\log\left(\prod_{\tau_1, \tau_2, Y^* \in T} P(R_\theta(\tau_+) > (R_\theta(\tau_-)))\right) \\ &= \sum_{\forall \tau_1, \tau_2 \in T} -\log\left(\frac{\exp \sum_{x \in \tau_+} r_\theta(x)}{\exp \sum_{x \in \tau_+} r_\theta(x) + \exp \sum_{x \in \tau_-} r_\theta(x)}\right) \end{aligned} \quad (8)$$

As we see, this is the cross-entropy loss derived by Bradley-Terry, and hence is our baseline loss function.

2.2 Richardson-Srikumar-Sabharwal Goodness Constraint

Next, we will formulate pairwise preferences as an implication, allowing us to use fuzzy logic and relax the implication into a differentiable function for gradient based optimization. This approach parallels the approach outlined by Richardson et al., which results in a perceptron-style loss function which bears striking resemblance to RRHF [? ?].

We begin by defining the predicate $IsGood(\tau)$ as the sigmoid of the cumulative reward of a trajectory τ , which describes as a degree of truth how good the given trajectory is:

$$IsGood(\tau) = \sigma\left(\sum_{x \in \tau} r_\theta(x)\right) \quad (9)$$

Here, $r_\theta(x)$ is defined as in Equation 2. This captures the idea that trajectories with large rewards are considered good; that is, large rewards have degrees of truthfulness close to 1 (i.e., $IsGood(\tau)$ is close to 1) and vice versa.

2.2.1 Relaxing the Implication With Fuzzy Logic Results In RRHF Loss. If we have the pairwise preference $\tau_+ \succ \tau_-$, we can define our implication as:

$$\alpha = IsGood(\tau_-) \rightarrow IsGood(\tau_+) \quad (10)$$

Which we can then relax using a t-norm of our choice. For example, if we choose to perform the relaxation under the product t-norm, we get:

$$[\alpha] = \min\left(1, \frac{IsGood(\tau_+)}{IsGood(\tau_-)}\right) \quad (11)$$

This imposes the preference that $IsGood(\tau_+)$ should be more true than $IsGood(\tau_-)$, which is exactly what we want under our proposed formalism. To get our loss function, we can take the negative log likelihood of Equation 11:

$$L_{RSS}(\alpha, \tau_+, \tau_-) = \max[0, \log(IsGood(\tau_-)) - \log(IsGood(\tau_+))] \quad (12)$$

2.2.2 We Can Further Constrain The Loss Landscape With The "One-True Constraint". Again, we can take a parallel approach to that offered by Richardson et al., adding further logical constraint [?]. We can introduce the "one-true constraint" in conjunction with L_{RSS} to retrieve a logical formula which closely resembles the fully constrained version of the Contrastive Preference Optimization [?] loss function when applied in conjunction with Equation 12:

$$\beta = IsGood(\tau_+) \vee IsGood(\tau_-) \quad (13)$$

If we again use the product t-norm, we can relax this boolean formula, and transform it into the negative log space to get a loss function.

$$L_{OT} = -\log(IsGood(\tau_+) + IsGood(\tau_-) - (IsGood(\tau_+) \cdot IsGood(\tau_-))) \quad (14)$$

2.3 Reward Penalty Loss

Through empirical tests we discovered that further constraining the loss landscape would be necessary to introduce a separation between the positive preference rewards and negative preference rewards when training with RSS (Section 2.2); we discuss this in depth in Section 5 but present the *reward penalty (RP)* constraint here.

The Reward Penalty (RP) enforces $IsGood(\tau_+) \wedge \neg IsGood(\tau_-)$, or that the preferred trajectory should be good and the dis-preferred trajectory should not. Taking the negative log of this gives RP loss:

$$-\log(IsGood(\tau_+) \cdot (1 - IsGood(\tau_-)))$$

2.3.1 While this is very similar to Bradley-Terry loss, there is an important distinction. This is our familiar logistic regression loss, similar to in 2.1. However, in BT loss, the input to the sigmoid differs. Here where R is the cumulative

reward of a trajectory,

$$\sum_{\forall \tau_1, \tau_2 \in T} -\log(IsGood(\tau_+) \cdot (1 - IsGood(\tau_-))) = \sum_{\forall \tau_1, \tau_2 \in T} -\log(\sigma(R_{\tau_+}) \cdot (1 - \sigma(\tau_-))) \quad (15)$$

While under the Bradley Terry Model:

$$\sum_{\forall \tau_1, \tau_2 \in T} -\log\left(\frac{\exp(R_{\tau_+})}{\exp(R_{\tau_+}) + \exp(R_{\tau_-})}\right) = \sum_{\forall \tau_1, \tau_2 \in T} -\log(\sigma(R_{\tau_+} - R_{\tau_-})) \quad (16)$$

2.4 Resulting Loss Function

We have one resulting loss function with four individually weighted components as follows:

$$L = \lambda_{RSS}L_{RSS} + \lambda_{BT}L_{BT} + \lambda_{OT}L_{OT} + \lambda_{RP}L_{RP} \quad (17)$$

3 Experimental Setup

We implement the proposed loss functions and evaluate agentic performance using the CartPole environment [?] and PPO. We investigate multiple formulations of loss functions using the derivations seen in Section 2 to train multiple reward models with both hard and soft reward routing during training (see Section 1.2). These are each compared to the Bradley-Terry baseline loss function with soft reward routing at training time and argmax routing at test time as seen in Kalra et al.'s paper [?] to use in PPO training.

3.1 Grid Search

To understand how the combinations of loss functions behave, we performed a grid search. Due to the large number of potential hyper-parameters and the resulting number of combinations, we limited our search to the inclusion factors for each loss function, the learning rate, and the DDT training routing options. For each DDT trained, we set the depth to 2.

Hyper-parameter	Set of Values
λ_{RSS}	$\{0, 1, 1e6\}$
λ_{BT}	$\{0, 1, 1e6\}$
λ_{OT}	$\{0, 1, 1e6\}$
λ_{RP}	$\{0, 1, 1e6\}$
Learning rate	$\{1e-4, 1e-3, 1e-2\}$
Routing Option	$\{\text{hard}, \text{soft}\}$

Table 1. The set of hyper-parameters and their corresponding values searched over.

3.2 Sigmoid Parameters

We introduced a shift parameter, s , into the sigmoid function used for the predicate $IsGood(\tau)$. This allows us to control the point at which an output value of 0.5 is produced from the sigmoid. We set $s = -2$, which shifts the sigmoid such that an input reward of 0 to $IsGood(\tau)$ has an output closer to 0, rather than 0.5 if the shift was not introduced.

$$\sigma(x) = \frac{1}{1 + e^{-x-s}} \quad (18)$$

4 Results

As seen in Table 2, the only model that has comparable performance to the BT baseline is the modified Logistic Regression loss function (RP) in conjunction with the One True Constraint and the Richardson-Srikumar-Sabharwal Goodness Constraint (RSS). While the models using hard reward routing (See Figure 5a and 4a) during training can learn to classify positive and negative rewards given preferred and dis-preferred trajectories, the resulting reward functions can not be used to train an agent that effectively balances the pole in the CartPole environment, receiving average rewards over five episodes of under 20. Furthermore, while the RSS hard reward model learns to classify the trajectories much better than RSS soft (See Figure 3a and 4a), the agentic performance is only marginally increased.

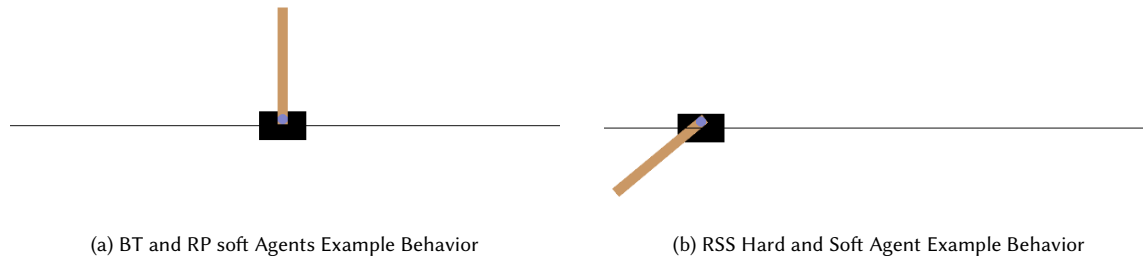


Fig. 1. Comparing BT and RP Soft Agents with RSS. The agents that use soft routing and either BT or RP perform well. The agents that use hard routing perform poorly.

Model	BT	OT	RP	RSS	Routing	Best Epoch	Best Val Acc	Avg Reward
BT soft (baseline)	1.0	0	0	0	soft	8	95.0	200
RSS OT RP hard	0	1,000,000.0	1.0	1.0	hard	2	86.5	18.8
RSS OT RP soft	0	1.0	1,000,000.0	1.0	soft	4	95.0	200
RSS hard	0	0	0	1.0	hard	3	85.0	16.8
RSS soft	0	0	0	1.0	soft	2	60.0	10.8

Table 2. Comparing Reward Models' Validation Accuracies Using Their Respective Loss Functions

5 Discussion

5.1 Diverging Rewards

While performing the DDT experiments using the RSS loss function, we noticed that we were able to achieve high validation accuracy but the agents trained on these DDTs performed poorly. Upon inspection of the rewards for the positive and negative preferences, we realized that the DDT was learning to assign a higher reward to the positive preference, but only just enough to satisfy the implication and minimize the loss. This can be clearly seen in Figures 3b and 4b, where we calculate the average reward for all of the true positive preferences and all of the true negative preferences, and their standard deviations respectively. If we create this same plot for the DDT trained using BT, it

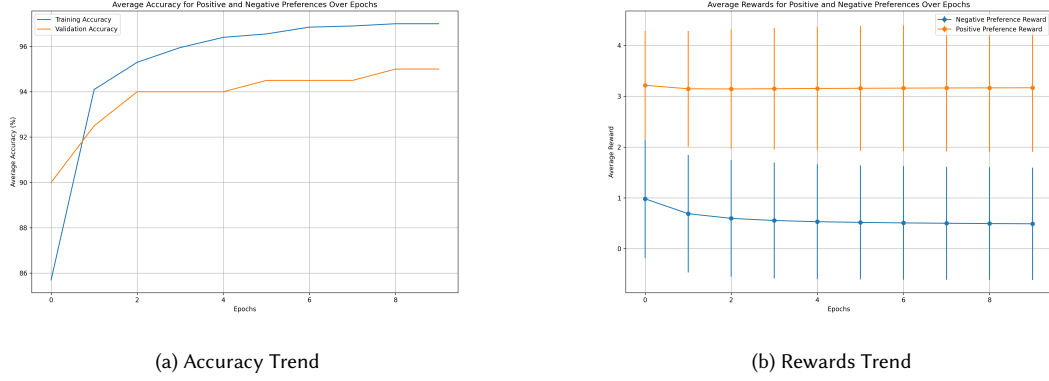


Fig. 2. Bradley-Terry Soft with a: Training (blue) and Validation (orange) Accuracy Over 10 Training Epochs; b: Average Positive Preference (Orange) and Negative Preference (Blue) Reward Prediction Over 10 Training Epochs

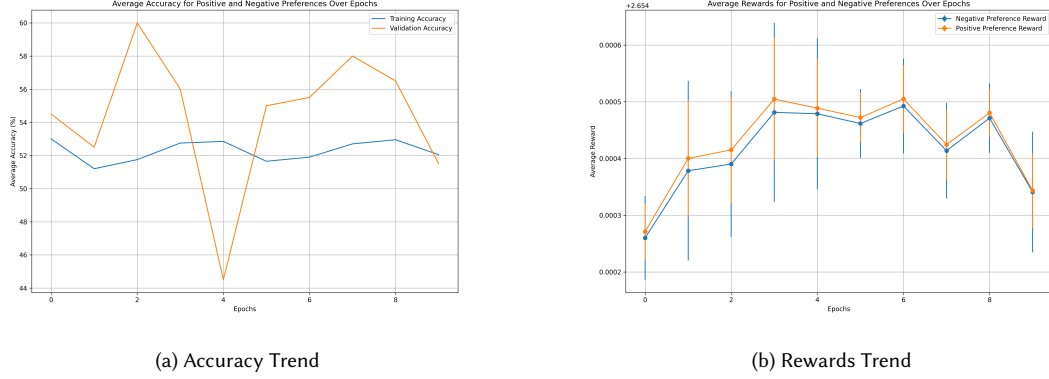


Fig. 3. RSS Soft with a: Training (blue) and Validation (orange) Accuracy Over 10 Training Epochs; b: Average Positive Preference (Orange) and Negative Preference (Blue) Reward Prediction Over 10 Training Epochs

becomes clear that as training progresses that the average positive preference reward and average negative preference reward begins to diverge (Figure 2b). In an attempt to replicate the reward trend of BT trained DDTs, we decided to introduce another constraint – namely the RP loss discussed in Section 2.3 – to encourage the DDT to introduce a difference in rewards. As seen in Figure 6b (RSS OT RP soft reward routing trained DDT), we are able to recover the diverging reward trend. For the same DDT loss function using the hard reward routing, we are unable to see this trend due to the OT factor dominating the loss function, and thus the RP penalty is negligible. In other experiments using hard reward routing where the RP factor contributes more to the loss (e.g., DDTs trained using RSS and RP alone), the RP factor causes the diverging reward trend that BT displays.

5.2 Hard Reward Routing; Poor Performance, But Promising Preliminary Findings

Originally, we tried to only use soft reward routing while training the reward DDTs, but quickly found that many of the loss function combinations simply could not learn to effectively apply the constraints. One-True (OT), and RSS,

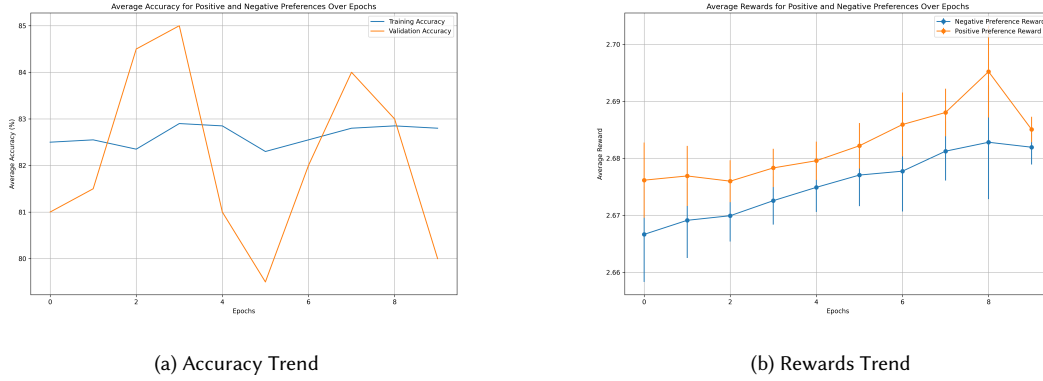


Fig. 4. RSS Hard with a: Training (blue) and Validation (orange) Accuracy Over 10 Training Epochs; b: Average Positive Preference (Orange) and Negative Preference (Blue) Reward Prediction Over 10 Training Epochs

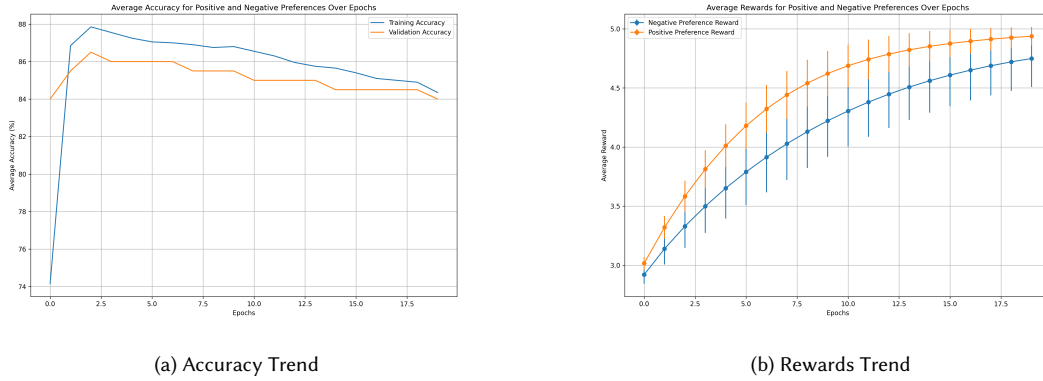


Fig. 5. RSS OT RP Hard with a: Training (blue) and Validation (orange) Accuracy Over 10 Training Epochs; b: Average Positive Preference (Orange) and Negative Preference (Blue) Reward Prediction Over 10 Training Epochs

specifically greatly struggled to effectively learn their respective constraints and minimize their corresponding loss functions. On the other hand, using hard reward routing greatly increased the training and validation accuracy, along with the resulting agents' rewards with both OT and RSS. Unfortunately, their performance compared to the logistic regression style loss functions are underwhelming with the experimental setup that we used.

With that being said, while we searched the hyper-parameter (HP) space for hard reward routing DDTs, there are many HPs that we did not investigate, such as tree depth, learning rates, sigmoid scale and shift, and normalization. Preliminary results show that further investigation of these aspects of the reward models may improve performance and lead to reward models that use hard routing and achieve better agentic performance.

We also obtained preliminary results while further tuning PPO HPs for the hard-routing reward models. For example, we trained one model with differing PPO HPs that used only RSS and RP that was able to get promising episodic rewards. However, due to compute and time constraints, we were unable to exhaustively search the space of HPs or robustly document and report these results.

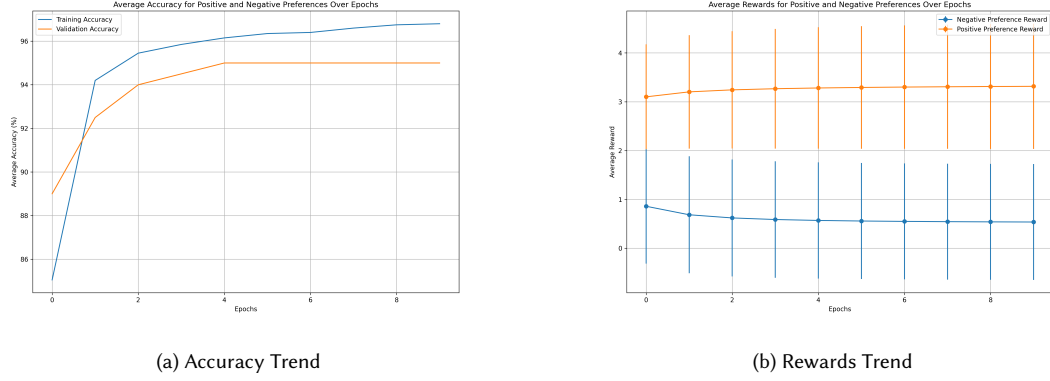


Fig. 6. RSS OT RP Soft with a: Training (blue) and Validation (orange) Accuracy Over 10 Training Epochs; b: Average Positive Preference (Orange) and Negative Preference (Blue) Reward Prediction Over 10 Training Epochs

Therefore, we believe that hard reward routing may prove to be a valuable strategy to help the DDTs effectively exploit the logical relationships that we have derived, and may indeed prove to be a valuable framework for interpreting complex reward modeling, but many challenges lie ahead in testing this hypothesis.

5.3 Future Work

Throughout working on this project, we formulated multiple new ideas to follow up with that we were unable to achieve due to the time constraints. We list the most interesting ideas here: 1) It is interesting to consider the number of logical constraints that can be relaxed and introduced into the loss function, and whether there is the potential for being able to perform a search over these loss functions or creating a DDT itself to learn which logical expression best suits the given pair of trajectories. 2) With a DDT, it seems as if the interpretable structure lends itself to the introduction of logical constraints in the internal nodes. 3) Due to the small number of parameters used in our DDTs, we are also interested in distilling knowledge from a larger model. This is particularly interesting for the DDTs trained using the hard reward classification model due to the enhanced interpretability.